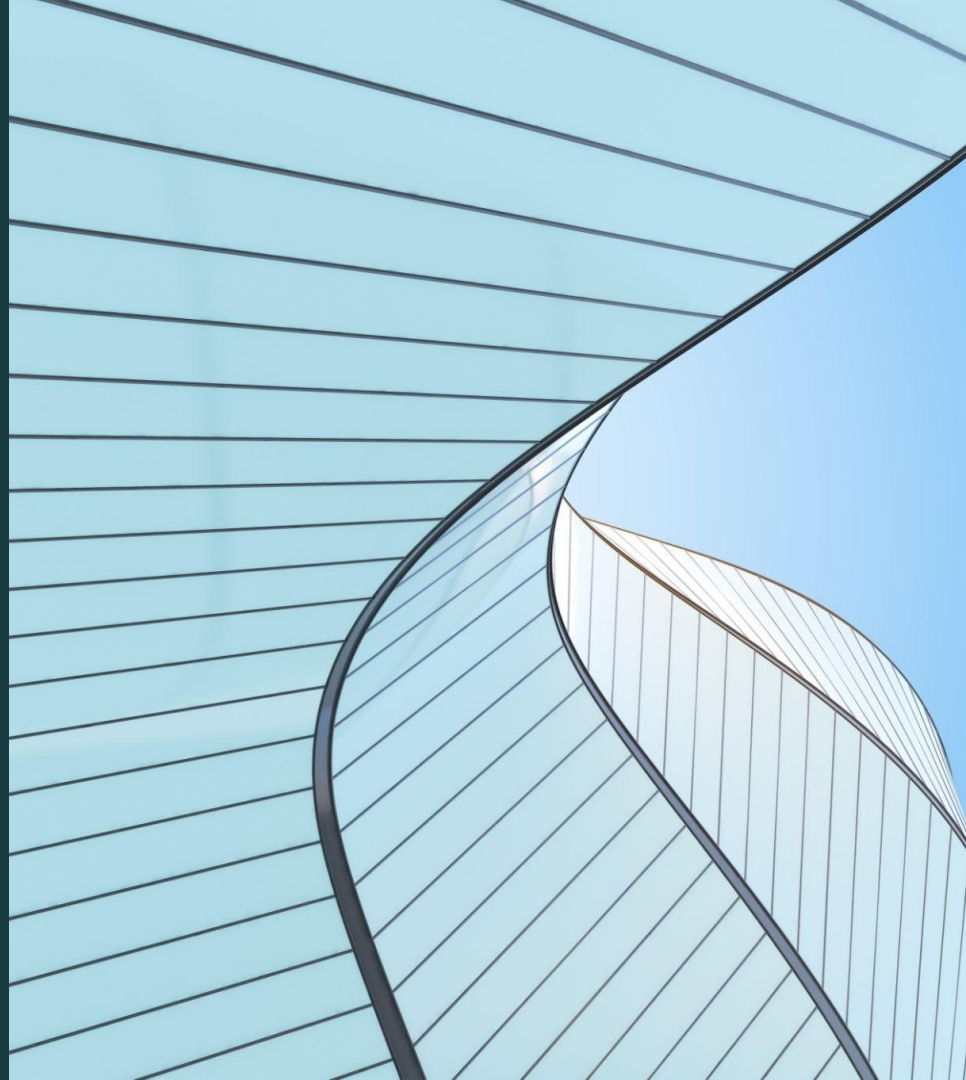
Daily Dev Mix

By Greg Lowman, Devang Pandey, Ian Martin, Matthew Aldridge, Vera Zaric, Dylan Long



What our Product Does

Problem

Spotify's activity playlists are generic.

They don’t reflect what you actually listen to during a session.

Who it's for

Anyone who listens to music during focused activities or daily routines.

Solution

1. Track your real listening behavior during a vibe session

2. Auto-generate a personalized playlist from what you actually heard.

Why us

Session-driven, activity-based playlists built from *your* taste

How it Works

1. Spotify OAuth login

Connect your Spotify account securely. No separate sign-up needed.

2. Start a vibe session

Pick an activity (Studying, Workout, Party) or create your own vibe.

3. Real-time song tracking

Every track plays in Spotify, we capture it live and display session stats as you listen.

4. Auto-create playlist

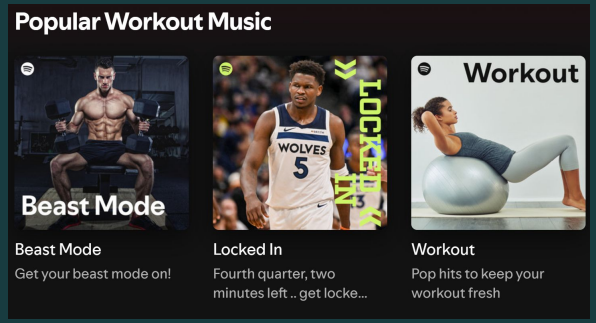
Ending a session creates a playlist instantly saved to your Spotify account.

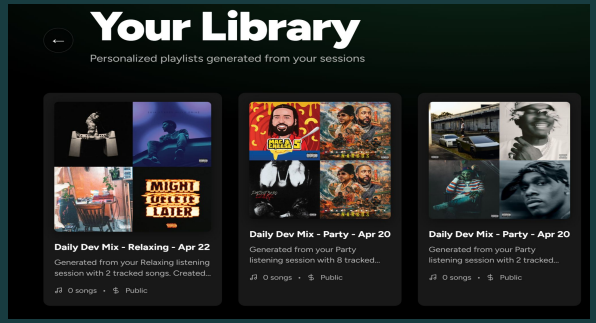
5. Playlist library

Browse every playlist generated by vibe, revisit past sessions, and track listening stats over time.

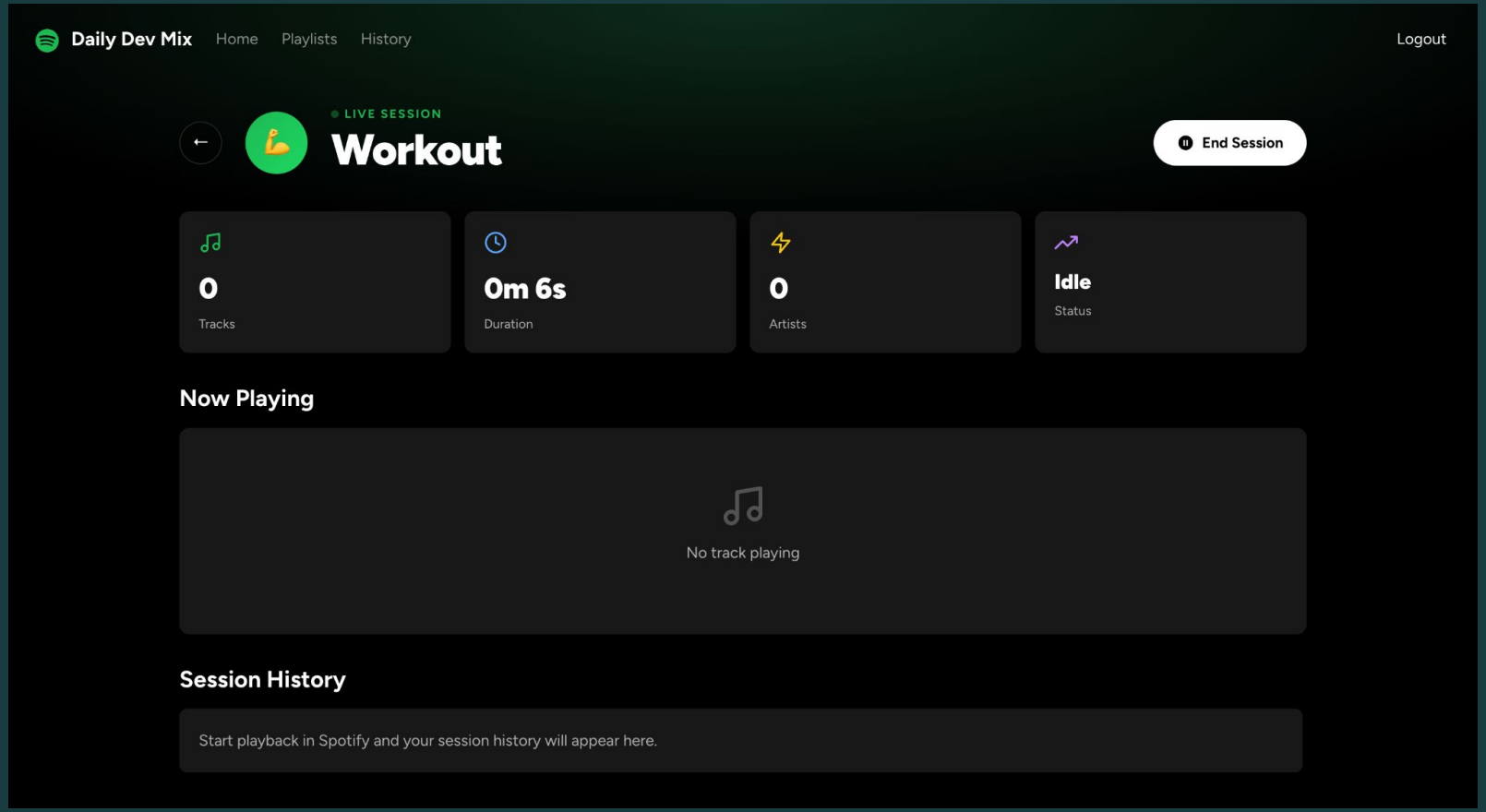
What make Daily-Dev-Mix unique?

Daily Dev Mix gives Spotify users truly personalized playlists, built from what they actually listen to, not what an algorithm guesses they might like. Currently on Spotify you can browse through various playlists generated by the app to attempt to curate music for a mood or event. While these playlists can be similar to what you may listen to, they do not always hit the mark. For instance, if a user typically listens to a certain genre and decides they want to put on a study playlist, Spotify may create a playlist of good study music based on the user genre preference. Our app would instead have the user pick a 'vibe' and while listening to music on Spotify, the Daily Dev Mix would retrieve the metadata via Spotify API. Using the artists the user listened, the app then generates a playlist tailored to that exact mood, based on your real listening history.



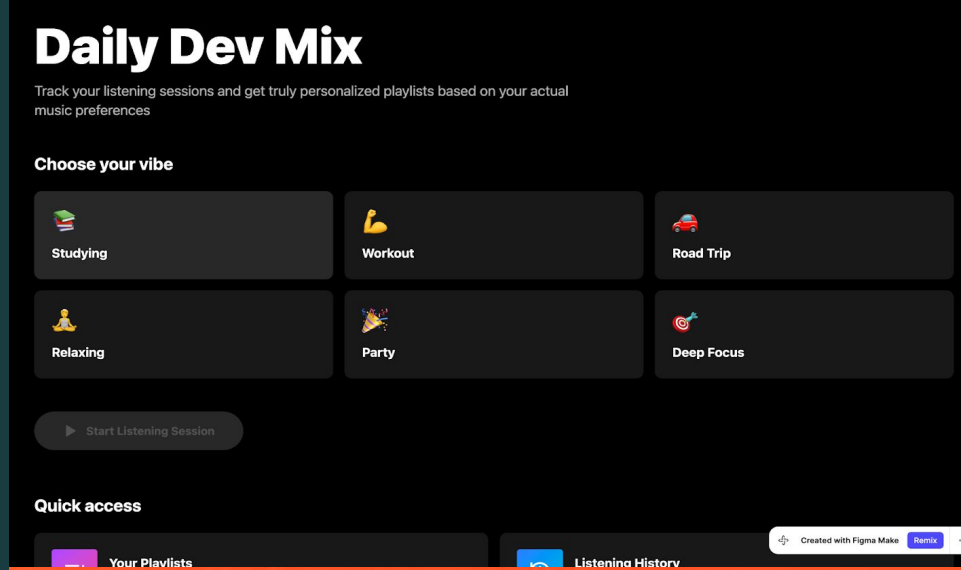


How it Works


Daily Dev Mix
Home
Playlists
History
Logout
LIVE SESSION
Workout
End Session
0
Tracks
0m 6s
Duration
0
Artists
Idle
Status
Now Playing
No track playing
Session History
Start playback in Spotify and your session history will appear here.

Wireframes + Handlebars:

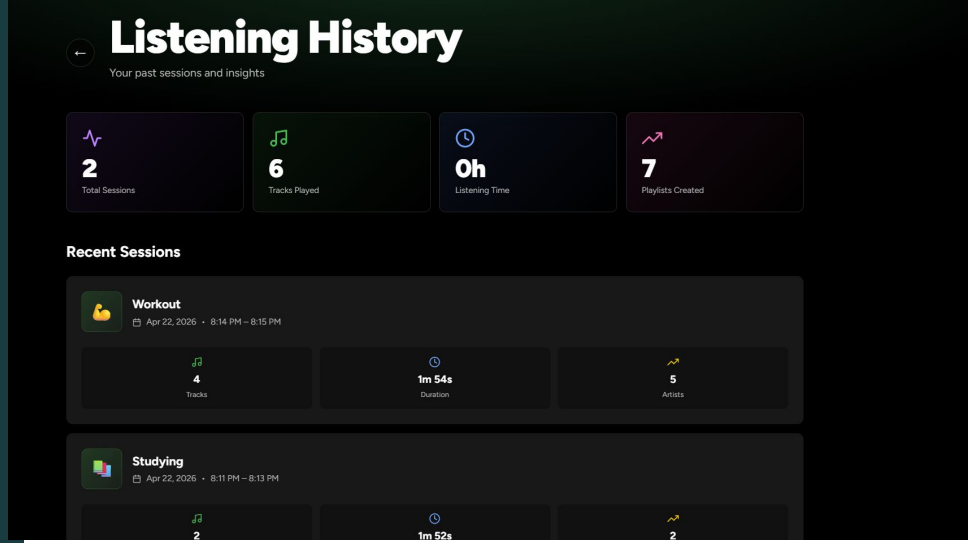
- Created initial Wireframes using Figma
- Transformed the design into working detailed UI using Handlebars
- Allowed for seamless integration and consistent design



handlebars

PostgreSQL Database:

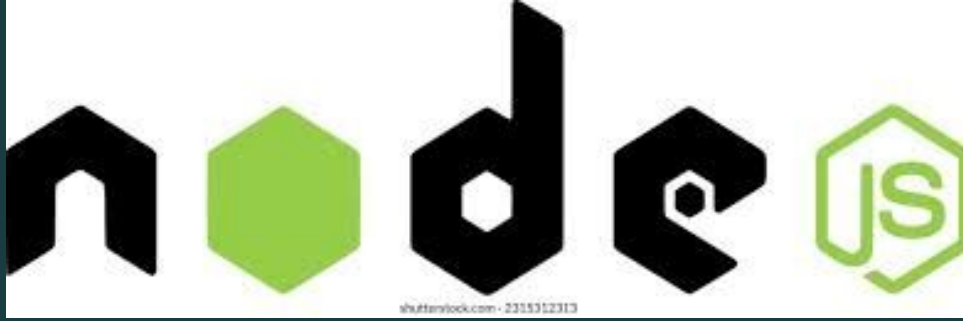
- Store User's Spotify ID + Listening Session Information + Categories
- Allowed for Custom Categories + History feature




```sql
CREATE TABLE if not exists sessions(
    session_id SERIAL PRIMARY KEY,
    user_id INT,
    category_id INT,
    start_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    end_time TIMESTAMP DEFAULT,
    song_count INT,
    artist_count INT
);
CREATE TABLE if not exists users (
    id SERIAL PRIMARY KEY,
    spotify_id VARCHAR(50) UNIQUE NOT NULL
);
```

PostgreSQL

Node.js

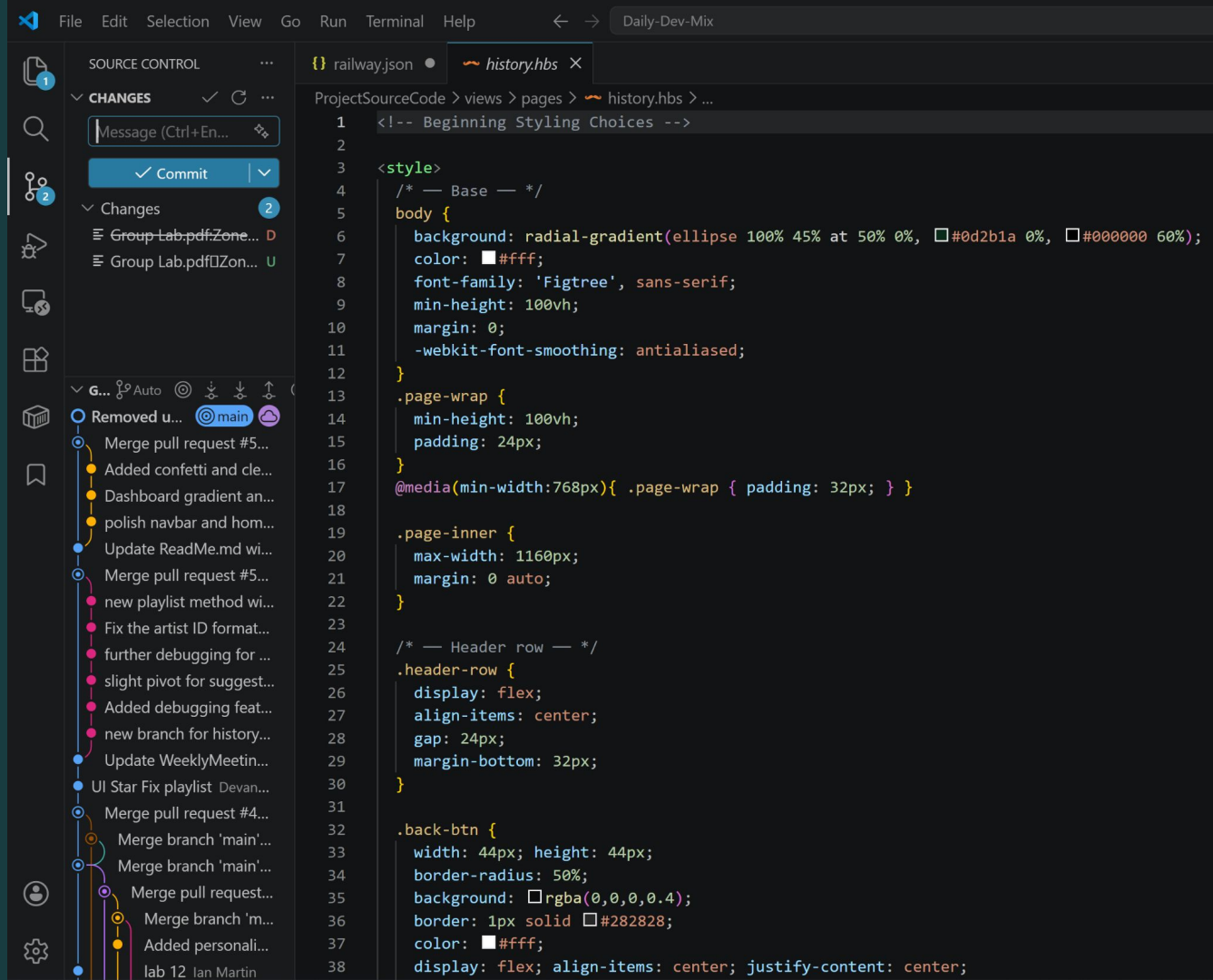


- Backend server that handles HTTP requests from frontend, processes user actions such as login, registration, and session creation
- Communicates with our PostgreSQL database
- Node.js is efficient in handling multiple user requests; allows multiple users to start or resume listening sessions simultaneously without slowing down the server

```
app.post('/api/session/start', authApi, async (req, res) => {
  const label = String(req.body.label || '').trim();
  const emoji = String(req.body.emoji || '🎵').trim() || '🎵';
  const forceRestart = Boolean(req.body.forceRestart);
```

IDE

The IDE used by the team was Visual Studio. Visual Studio has some perks over other IDE's and it is primarily used here at CU so it was easy to use what we already know. The IntelliSense feature within V.S continues to be incredibly useful for quickly writing code. We were also able to link github directly to the IDE which can be useful for consistently committing or pulling from main.



Spotify API

OAuth Flow

1. Redirect → Spotify consent
2. Callback with auth code
3. Exchange → access token + refresh token
4. Stored in express-session
5. Auto-refresh on expiry

Scopes requested

- user-read-private
- user-read-currently-playing
- user-read-recently-played
- playlist-modify-public
- playlist-modify-private

Key Endpoints

- GET /me/player/currently-playing
- GET /me/player/recently-played
- GET /me/playlists
- POST /me/playlists
- POST /playlists/{id}/items

Error handling

- 401 → refresh token / re-login
- 403 → scope/dev-mode guidance
- 429 → rate limit retry message

SESSION → PLAYLIST FLOW

Start session — User picks vibe › Poll tracks — currently-playing › Build session — append to state › End session — backfill recent plays › Create playlist — POST + insert URIs › Opens in Spotify — saved to account

Testing + Cloud Hosting

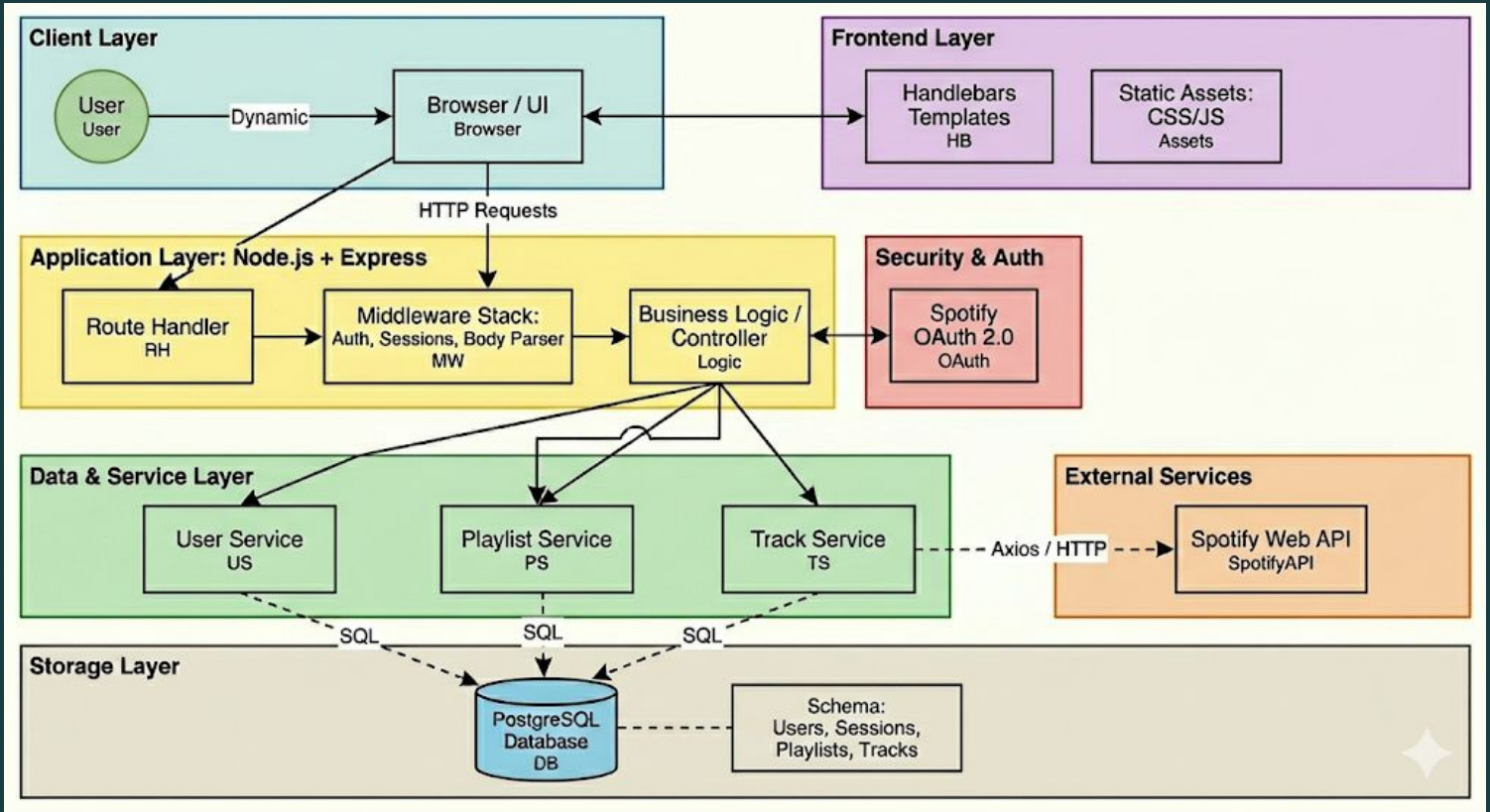
- Unit Tests of routes using Chai
- Deployment on Railway with each commit
- Allowed for better acceptance testing using the same environment

```
it('Redirects unauthenticated page requests to /login', async () => {
  const { res } = await invokeRoute('get', '/home');

  expect(res.statusCode).to.equal(302);
  expect(res.headers.location).to.equal('/login');
});

it('Rejects unauthenticated API session starts', async () => {
  const { res } = await invokeRoute('post', '/api/session/start', {
    body: { label: 'Studying', emoji: '📚' },
  });
```

Railway

chai

Architecture Diagram



Project Management

Meetings

We had meeting whenever we had major merge conflicts of disagreements about where the project was headed. The meetings were mostly in person after lecture, but a few were on Zoom later in the day.

Tracking Tools

Our main method of communication was through discord where much of the smaller and less important conversations were had. We used the project board on Github to track our progress and who is doing what.

Challenges

- Communication

  Our team initially struggled with clear communication, which led to issues like duplicate work and uncoordinated pull request merges. For example, multiple people would unknowingly work on the same task, and some would merge their own PRs without notifying others. To address this, we established a set of ground rules. We agreed to strictly follow the project board for task management, require that all pull requests be reviewed by a teammate before merging, and prohibit self-merging. We also committed to maintaining open communication through our Discord channel and approaching collaboration with flexibility and respect for each other's ideas.

- Correctly using AI

  For many of us, this was our first time incorporating AI into a workflow that directly contributed to building a finished product. It was easy to rely on AI without fully understanding what it was producing. At one point, much of our project ended up being essentially "vibe coded" in React Native, even though no one on the team had experience with it. This forced us to restart and reevaluate our approach. From that experience, we set a guideline to use AI more intentionally and thoughtfully, ensuring we understand and validate its output rather than relying on it blindly.

Future Scope/Enhancements

- More Depth

  Right now, Daily Dev Mix could feel more like a single feature than a fully developed SaaS product, since its core function of generating playlists may not be enough to keep users consistently engaged or make it part of their daily listening habits. To address this, a key future goal is to expand its feature set so it becomes more of a central hub for music discovery and interaction, offering enough value and unique functionality to stand on its own rather than simply acting as an extension of Spotify. An example of something to take this app to the next level is the addition of gamification/stats of the user's listening history. Similar to a spotify wrapped, but showing up year round instead of every new years.